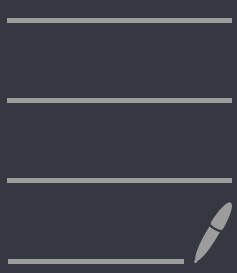
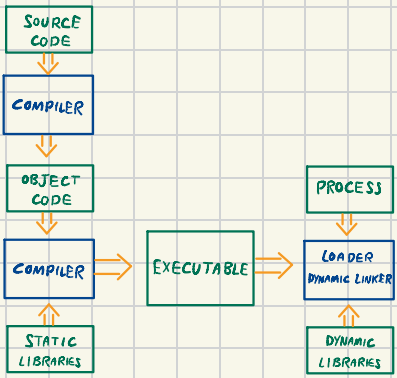
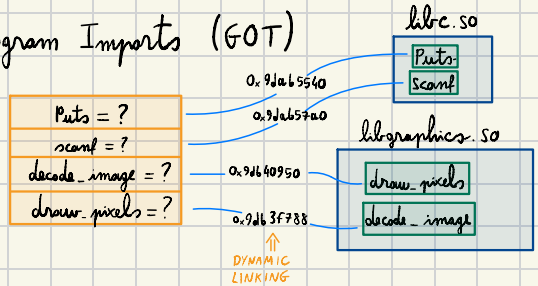
PLT & GOT



Abusing ELF Dynamic Linking

Grazie ad alcune vulnerabilità è possibile scrivere dati arbitrari in indirizzi di memoria arbitrari (es. out-of-bound accesses).
Un metodo per sfruttare questo exploit consiste nell'abusare dei meccanismi interni del linking dinamico ELF.

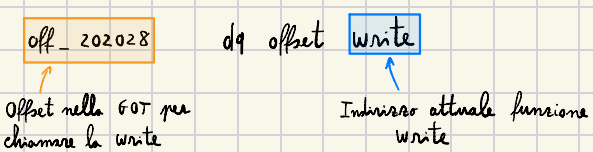




- GOT permette ai codici compilati di eseguire correttamente, indipendentemente dall'indirizzo di memoria dove vengono caricati a runtime

Global Offset Table (GOT) in IDA

Grazie ad IDA è possibile ispezionare tutti gli offset delle funzioni nella GOT:



Procedure Linkage Table (PLT)

Per raggiungere la GOT table è necessario passare prima per la PLT

- Collezione di Trampolini (uno per ogni import)
  - Il programma chiama una entry PLT
  - L'entry PLT esegue delle JMPS nella GOT

- Perché ci sono questi reindirizzamenti extra?
  - collegamento dinamico ritardato e complicato
  - Non-PIE (Position Independent Executable) dynamically linked programs.

ES: JMP `CS:off_202028` → jumpa alla GOT entry corrispondente

GOT Hijacking

1) Sovrascrittura di una GOT entry tramite delle corruzioni della memoria
2) Portare il programma a chiamare la funzione PLT corrispondente
   - la chiamata verrà instradata tramite la GOT
3) ?!?
4) Controllo del PC (Program Counter)

ES:
```
char buf[100];
scanf("%99s", buf);
puts(buf);
```

con un input adeguato è possibile impostare come valore di buf "/bin/bash" e overwriteare il return ad una funzione "system" esistente.
Così verrà eseguito system("/bin/bash") al posto di puts(...)

Lazy Linking

Problema: trovare i simboli è un processo lento, se essi venissero caricati tutti all'avvio del programma, il tempo di avvio sarebbe molto lungo.

Dato che non tutti i simboli vengono usati contemporaneamente, è possibile sfruttare il Lazy Linking per postporre la symbol resolution a quando vengono usati

Questo significa che per un attaccante è necessario corrompere una entry della GOT per una funzione.
Se la PLT trova l'entry della GOT della funzione vuota:
- risolverà il suo simbolo e otterrà il vero indirizzo della funzione
- altrimenti chiamerà direttamente la funzione all'indirizzo specificato nella GOT

⇒ Se scriviamo nella GOT l'indirizzo della nostra funzione malevola,

il programma la chiamerà.

Nota: è meglio sovrascrivere l'entry di una funzione che è già stata chiamata almeno una volta.

RELRO: GOT Attacks Mitigation

RELRO = RELocation Read-Only

- Full RELRO: tutta la GOT è Read Only
  - + rende tutta la GOT RO ⇒ immune ad Hijacking
  - − incompatibile con il Lazy Linking

- Partial RELRO: una parte della GOT è RO (quella relativa alle variabili globali) ma le funzioni hanno comunque problemi
  - + compatibile con il Lazy Linking
  - − l'hijacking è comunque possibile